Week 1 report | Cloud computing, AWS and IAM security

Davine Technologies Internship Project, AWS Cloud internship week 1 | Oseni Sakariyau Oluwadamilare | Exercise week of 10 August 2026, rebuilt and re-verified 13 November 2026

This is the short report the week 1 brief asked for, written in my own words. Every AWS-specific claim was checked against AWS documentation or against my own account on 13 November 2026; the checks are listed in ai-verification-log.md.

1. Cloud computing

Cloud computing is renting computing (servers, storage, databases, networking, software) over the internet and paying for what you use, instead of buying and running the hardware yourself. The provider owns the data centres; you ask for capacity through a console or an API and get it in minutes.

Why teams move to it:

- **No up-front hardware cost.** Spending moves from buying servers (capital expense) to paying monthly for use (operating expense).
- **Elasticity.** Capacity grows and shrinks with demand, so you stop paying for idle servers sized for the busiest day.
- **Speed.** A new environment is an API call away, which is what makes infrastructure as code (like the Terraform in this repository) possible.
- **Global reach.** The same account can deploy close to users on several continents.
- **Reliability and security at scale.** The provider runs redundant facilities and security teams that most companies could not afford on their own.

2. Service models: IaaS, PaaS and SaaS

The models differ in how much of the stack the provider runs for you.

	IaaS	PaaS	SaaS
You get	virtual servers, networks, disks	a platform to run your code	a finished application
You manage	OS, runtime, app, data, access	your code, data and access	your data and who has access
Provider manages	hardware, virtualisation, facilities	everything up to the runtime	everything
Example	Amazon EC2, Amazon VPC	AWS Elastic Beanstalk, Amazon RDS	Amazon QuickSight, Microsoft 365
Trade-off	most control, most work	less control, much less work	least control, no ops work

A useful way to remember it: the further right you go, the less you patch and the less you can change.

3. Deployment models: public, private and hybrid

- **Public cloud:** the provider's shared infrastructure, used over the internet by many customers who are isolated from each other (AWS, Azure, Google Cloud). Cheapest to start and fastest to scale.
- **Private cloud:** cloud-style self-service on infrastructure used by one organisation only, in its own data centre or hosted. Chosen for strict control or regulation, at the cost of owning the hardware.
- **Hybrid cloud:** both, connected, with workloads placed where they fit best. A bank might keep its core ledger on premises and run its mobile app back end on AWS, linked by AWS Direct Connect or a VPN.

The deployment model answers *where* the infrastructure runs; the service model answers *how much of it you manage*. They are independent: you can run IaaS in a private cloud or SaaS from a public one.

4. AWS global infrastructure

- **Regions** are separate geographic areas, each with its own copy of most services. Data stays in the Region you choose unless you move it. On 13 November 2026 my account could see 34 Regions, 17 of them enabled; the rest, such as `af-south-1` (Cape Town), are opt-in.
- **Availability Zones (AZs)** are one or more data centres inside a Region, with separate power and networking, a few kilometres apart and joined by low-latency links. `us-east-1` has six (`us-east-1a` to `us-east-1f`). Spreading an application over two or more AZs is how it survives a data-centre failure.
- **Edge locations** serve Amazon CloudFront and Route 53 close to users. **Local Zones** and **Wavelength Zones** push compute nearer to cities and 5G networks; `us-east-1` has 14 of them listed for my account.
- **Global services** such as IAM are not tied to a Region, which is why the IAM resources in this lab exist account-wide.

5. Free Tier, Management Console and Billing Dashboard

- **Free Tier.** Accounts opened under the current sign-up get **US$100 in credits, plus up to US$100 more** for trying key services, on a **Free plan that lasts six months** or until the credits run out, whichever is first; upgrading to the Paid plan keeps the credits. The older "12 months free" description no longer matches what new accounts get. AWS's own FAQ is inconsistent on one detail (it also says upgraded credits expire after 12 months), so I note that rather than guess.
- **Management Console** is the web interface for every service. I used it to confirm and screenshot everything Terraform created.
- **Billing and Cost Management** shows month-to-date spend, forecasts and the bill. **AWS Budgets** sends alerts before money is spent; my account already had two (US$1 and US$40 a month), so this lab adds no third one.

6. IAM: identity and access management

IAM decides **who** can sign in (authentication) and **what they may do** (authorisation) in an AWS account. It is free and global.

The rule that explains every result in this project: **every request is denied by default, an Allow opens it, and an explicit Deny always wins over any Allow.** (The one exception is the root user, which has full access, which is why it must be locked away with MFA and not used day to day.)

7. IAM users, groups and roles

	What it is	Credentials	Use it for	In this project
User	one person or app with a fixed identity	long-term: password, optional access keys	a person who signs in to one account (in production, prefer IAM Identity Center)	`davine-wk1-developer-01`, no access keys
Group	a collection of users	none; it only carries policies	giving the same permissions to everyone in a team	`davine-wk1-developers`
Role	an identity that is *assumed*, not owned	temporary, issued per session	AWS services, apps, other accounts and federated people	`davine-wk1-ec2-app`, trusted only by EC2

Permissions go on the group, not on each person. A new hire is onboarded by adding them to the group and offboarded by removing them; nobody's permissions drift.

The role's use case: an application on EC2 needs to read its config from S3. Instead of copying access keys onto the server, the instance gets the role through an **instance profile**, and AWS hands it temporary credentials that rotate automatically. A stolen server disk then holds no reusable keys.

8. IAM policies

A policy is a JSON document of statements, each with an **Effect** (Allow or Deny), **Actions**, **Resources** and optional **Conditions**.

- **Identity-based policies** attach to a user, group or role (all four in this repository). **Resource-based policies** attach to a resource, such as an S3 bucket policy or a role's **trust policy**, which says who may assume the role.
- **AWS managed** policies are written by AWS (`AmazonSSMManagedInstanceCore` here); **customer managed** policies are written by you and reusable (the three `davine-wk1-*` policies); **inline** policies live inside one identity and are best avoided.
- **Least privilege:** grant only what the job needs. The developer policy allows starting and stopping only instances tagged `Team=developers`, and S3 access only to `davine-wk1-dev-*` buckets.
- **Conditions make policies smart.** `require-mfa` denies everything except setting up MFA and changing a password whenever `aws:MultiFactorAuthPresent` is false, so a stolen password alone is useless.

9. Multi-factor authentication (MFA)

MFA adds something you have (an authenticator app, a passkey or a hardware key) to something you know (the password). AWS recommends it for the root user and for every person who signs in. My account's root user has MFA enabled (`AccountMFAEnabled: 1` from `aws iam get-account-summary`), and the developer group cannot do anything useful until its members register a device.

10. The shared responsibility model

AWS describes the split as **security *of* the cloud** (AWS) versus **security *in* the cloud** (the customer).

- **AWS** protects the hardware, software, networking and facilities that run its services.

- **The customer** is responsible for what they put in and configure: their data and its encryption, IAM users and permissions, security group rules, and, for IaaS such as EC2, the guest operating system patches and the software they install.
- **The line moves with the service model.** On EC2 the customer patches the OS; on abstracted services such as S3 or DynamoDB, AWS runs the infrastructure, OS and platform, and the customer is left with their data and who can access it.

Everything this lab builds (MFA, the password policy, least-privilege groups, a role instead of keys) sits on the customer's side of that line. AWS will never configure it for you.

Sources

- AWS, *Shared Responsibility Model*: https://aws.amazon.com/compliance/shared-responsibility-model/
- AWS, *Policy evaluation logic*: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
- AWS, *Security best practices in IAM*: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
- AWS, *IAM roles*: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
- AWS, *AWS Free Tier* and its FAQs: https://aws.amazon.com/free/
- My account, 13 November 2026: `aws ec2 describe-regions`, `aws ec2 describe-availability-zones`, `aws iam get-account-summary`